Chapter 2: SOFTWARE REQUIREMENTS SPECIFICATION

The Software Requirements Specification (SRS) is a document that states what a software system must do and the qualities it must have, written so that developers, stakeholders and testers share one agreed description of the product [11]. For MezFlow it acts as the master plan for the engineering team, because it records the targeted capabilities, performance targets and limits that govern the whole development cycle of the dining, kitchen and billing platform [11, 23].

The SRS is important within the software development life cycle for three reasons. It gives a clear understanding of the project boundaries that were introduced in Chapter 1, it bridges the communication gap between the developers, the supervisor and the restaurant stakeholders, and it gives the later design, verification and testing phases a baseline to check the finished system against [11, 23]. Because MezFlow follows Agile Scrum, the requirements below form the initial product backlog and will be refined sprint by sprint, with changes agreed with the supervisor at each sprint review [13].

2.1 Introduction

The SRS is a foundational pillar of the MezFlow project. It gives an in-depth breakdown of the features, operational capabilities and performance targets that the system must offer to small and medium dine-in restaurants, and it acts as a formal understanding between the three-member development team, the supervisor and the stakeholders [11]. MezFlow connects customers, kitchen staff, service staff, cashiers and managers through one real-time platform, so the specification has to describe each of these groups and the way their work meets in a shared table session, an order and a bill [23].

The purpose of this SRS is to present a transparent and clear view of the system needs and to create one shared vision among everyone involved in production. A well-structured document improves coordination between engineers and stakeholders and reduces confusion, and its value lies in making sure that the final platform matches the stated needs and the expectations of the restaurant [11, 23].

2.1.1 Document Scope

The scope of this document is to lay out a complete framework for the MezFlow system, to serve as a reference for developers, managers and stakeholders, and to guarantee a shared view of its purpose, structure and operational needs [11]. It starts with the overview of the platform goals from Chapter 1, namely connected ordering, kitchen visibility, delay detection and an explicit table lifecycle. It then breaks the system down into functional requirements for the twelve modules, for example account control, the QR menu, the shared table cart, the kitchen display, billing and analytics, and into non-functional requirements such as performance, security, usability and scalability [11, 34].

The document also describes the technology that the requirements rely on, which is a customer Progressive Web App, a kitchen display web application and a management console, all served by a REST and WebSocket backend with a PostgreSQL database [4, 8, 25, 30]. User permissions are grouped into five roles, the customer, kitchen staff, service staff, cashier and manager, each with its own access limits [9]. The Agile Scrum model with two-week sprints that governs the work was described in Chapter 1 and is not repeated here [13].

The SRS further covers the main user journeys, from scanning a table code and building a shared order to following the order, requesting the bill and confirming that the table has been cleaned. It records the security requirements for customer contact details and payment records, the requirements for testing through unit, API, integration, load and usability checks, and the requirement-gathering techniques and time frame used for this phase [11, 35]. Items that Chapter 1 placed out of scope, such as delivery marketplace integration, accounting or payroll, native mobile applications and automatic sensing of table cleanliness, are not specified here, and future enhancements are noted only as extension points so that the platform can adapt to new needs [11].

2.1.2 Audience

MezFlow is tailored for a diverse audience, and each group has distinct goals. The first group is the restaurant owners and managers, who need fewer ordering mistakes, visibility into which orders and tables need attention, and one console for the menu, tables, staff and reports [11, 12]. The second group is the kitchen staff, who need one organized and prioritized queue instead of scattered paper tickets during a rush, and the service staff and cashiers, who need a live table view, service-request notifications and an accurate itemized bill [11].

Customers are the third group, from solo diners to large tables. They want to order at their own pace in their browser without installing an application, to share one order with the people at their table, to understand unfamiliar dishes in English or Urdu, and to track when their food will arrive [4, 11]. For these readers the SRS explains what the customer experience will contain, so that they can confirm that it matches their expectations.

The document is also written for the project supervisor and evaluators, who use it to judge whether the scope is achievable within the Final Year Project timeline, and for the development team, which uses it as the reference for design, implementation and testing [11, 13]. Testers in particular will derive their test cases from the measurable targets stated in the functional and non-functional requirements [11, 23].

2.2 Functional Requirements

The functional requirements of MezFlow define the precise operations and capabilities that the system must deliver to satisfy its core purpose. They cover account control, the digital menu, collaborative ordering, the kitchen display, real-time synchronization, preparation tracking, the grounded AI explanation, billing, table lifecycle, administration, service

requests and analytics, and they are written as testable statements in the form "the system shall" [11].

2.2.1 Functional Requirements

Each requirement below belongs to one of the twelve modules introduced in Chapter 1 and is numbered so that it can be traced to design and test cases [11, 23].

1. User Registration and Login (Module 1): The system shall allow a user to sign up and log in with a phone number or an e-mail address, and it shall verify the user with a one-time password. After login the system shall issue a JSON Web Token session [29]. Users shall be able to update their profile, including the preferred language (Urdu or English) and notification preferences [11].

2. Role-Based Access and Audit Log (Module 1): The system shall support five roles, namely customer, kitchen staff, service staff, cashier and manager, and shall show each user only the screens and actions that the role permits [9]. Every staff action, such as a menu change, an order override or a bill approval, shall be written to an audit log with the user, time and action [11].

3. Digital Menu and QR Table Access (Module 2): Each table shall have its own QR code that opens the digital menu directly in the browser without installing an application [3, 4]. The menu shall show categories, item descriptions, images, prices and live availability, and shall support structured modifiers such as spice level, quantity, size, add-ons, ingredient removal and notes. A price or availability change made by the manager shall appear in the customer menu without the customer reloading the app [8].

4. Collaborative Table Ordering (Module 3): Several diners who scan the same table QR code shall join one shared table session. The shared cart shall record which diner added each item, shall update in real time for every diner at the table, and shall let the table submit one order for the kitchen [8, 11].

5. Kitchen Display System (Module 4): The kitchen display shall be a single-screen web application that shows every active order as an independent card with the order number, table number, items, modifiers, elapsed time and status. Kitchen staff shall be able to filter the cards by status (New, Preparing, Ready, Delayed) and by kitchen category such as Grill, Fry, Hot Kitchen and Beverage, and the screen shall run in an ordinary browser on a tablet, laptop or low-cost computer [6, 7].

6. Order Lifecycle and Real-Time Synchronization (Module 5): The system shall move each order through the states New, Confirmed, Preparing, Ready and Dispatched, and it shall allow only the transitions that this state machine permits [28]. Every state change shall be pushed over WebSockets to the customer app, the kitchen display and the management console, and customers shall see their order status without refreshing the page [8]. Simultaneous orders from different tables shall not interfere with each other.

7. Preparation-Time Tracking and Delay Detection (Module 6): The manager shall be able to configure an expected preparation time for each menu item. The system shall calculate the elapsed time from the moment preparation starts and shall flag the order as delayed on the kitchen display when the elapsed time exceeds the expected time. A customer-facing category such as Main Course shall be mappable to an internal kitchen station such as Hot Kitchen [22].

8. AI-Assisted Dish Explanation (Module 7): An "Explain This Dish" button shall generate a natural-language explanation in English or Urdu from the fields that the restaurant has supplied, namely ingredients, flavor, texture, heat level and portion size. The explanation shall use only those fields and shall not state facts that the restaurant did not provide, and the manager shall control which fields are supplied for each dish [10].

9. Billing and Payment Workflow (Module 8): A customer shall be able to request the bill from the app. A cashier shall review the bill, record payment as cash or card, and close the digital session, and the closure shall be recorded in the audit trail. The backend shall recompute every price from the stored menu and shall never trust a price sent from the customer device [11, 35].

10. Table Lifecycle Management (Module 9): The system shall keep the table state separate from the order state and shall move each table through Available, Occupied, Awaiting Cleanup and Available again [28]. Payment shall close the digital session without freeing the table, and the table shall return to Available only after a staff member confirms cleanup. Service staff and managers shall have a live table-status view [8].

11. Management Console (Module 10): The manager shall be able to create and edit menu items, prices, availability, categories and preparation times, to create tables and generate their QR codes, to monitor active sessions, to assign roles to staff, and to set restaurant-level options such as taxes, service charges and operating hours [9, 11].

12. Service Requests and Notifications (Module 11): A customer shall be able to request a waiter, a water refill or other assistance from the app. The system shall notify the customer about order status, bill readiness and acknowledgement of a request, using in-app notifications and push notifications where the browser supports them, and service staff shall see a queue of requests sorted by table and waiting time [4, 24].

13. Analytics and Operational Records (Module 12): The system shall show a dashboard of orders, average preparation time and delayed-order rate by date and category, shall keep an operational audit trail of order creation, state changes, payments and table-status changes, and shall let a manager export reports for review [11, 30].

14. Error Handling and Alerts: The system shall handle problems such as a lost network connection, a missing required choice or an unavailable item by showing clear and helpful messages. When the WebSocket connection drops, the client shall reconnect automatically and resynchronize its state with the server, so that the cart, the order and the table status are correct again [8, 24].

2.3 Non-Functional Requirements

Non-functional requirements describe how well the system must perform its functions and under which constraints it must operate [11]. For MezFlow the most important ones are speed and correctness of the real-time layer, security of accounts and payments, and usability for staff who work under time pressure. Where possible they are written with measurable targets so that they can be verified in testing [11, 34].

2.3.1 Software Quality Attributes

The quality attributes below follow the product quality model of ISO/IEC 25010, which groups the properties that decide how well a software product meets the needs of its users [34].

1. **Performance:** The customer app shall load its first screen in under 3 seconds on a typical mobile connection. Order, kitchen and table-status changes shall reach every connected client within 2 seconds under normal network conditions. The system shall handle at least 50 concurrent WebSocket connections and at least 10 simultaneous table orders submitted within a 2-minute window without state corruption or interference between orders [4, 8].

2. **Reliability and Availability:** The system shall be available during restaurant opening hours, shall recover from a dropped connection by automatic reconnection and state resynchronization, and shall keep the server as the single source of truth for carts, orders, tables and prices, so that a lost client never causes lost data [8, 24].

3. **Security:** Accounts shall be protected by one-time password verification and token-based sessions, and every screen and action shall be restricted by role [9, 29]. All prices shall be validated on the server, all input shall be validated, and the design shall consider the common web-application security risks listed by OWASP [35].

4. **Usability:** The interfaces shall be simple, filter-driven and available in English and Urdu, so that diners can order without help and kitchen staff need only brief training. Usability shall be evaluated with the System Usability Scale, with a target score of at least 70 from at least 10 participants [14].

5. **Maintainability:** The system shall be built as separate modules with a documented REST and WebSocket interface, so that new features can be added without rewriting existing ones, and it shall be covered by automated unit, API and integration tests that run on every code change [25, 26].

6. **Scalability:** The data of each restaurant shall be kept separate, so that more tables, locations, languages and payment methods can be added later without a redesign [23, 30].

7. **Compatibility and Portability:** The customer app and the kitchen display shall run in current mobile and desktop browsers and shall not require any proprietary hardware or installation [4, 31].

8. **Accessibility:** The interfaces shall follow the guidance of the Web Content Accessibility Guidelines 2.2, whose principles are that content is perceivable, operable, understandable and robust, for example through readable text size and contrast and full keyboard operation of staff screens [36].

9. **Data Integrity and Auditability:** Orders and tables shall change state only through their state machines, and every state change, payment and staff action shall be recorded in the audit trail, so that records can be reviewed after service [11, 28].

10. **Information Clarity:** Prompts, statuses and notifications shall use plain wording, so that every user always understands the current state and the next step [11].

2.3.2 Other Non-Functional Requirements (e.g. Legal, Platform etc.)

The success and trustworthiness of a restaurant platform depend on constraints that sit behind the visible features. On the legal side, the system stores customer contact details, order history and payment records, so it must protect them and handle them in line with the data-protection and consumer rules that apply to the restaurant, and the exact legal requirements for deployment in Pakistan are still to be confirmed with the supervisor and the pilot restaurant [11, 35]. The AI feature must not present invented information about dishes, because wrong allergen or ingredient statements could harm customers, which is why it is limited to the restaurant-supplied fields [10]. Taxes and service charges are configured by the restaurant and are not hard-coded.

On the platform side, the customer app is a Progressive Web App, the backend is written in Python with FastAPI and uses PostgreSQL, and the front ends use React and Next.js with Tailwind CSS [4, 26, 30, 31]. The system is deployed with Docker on a cloud server behind a web server, and it needs a working internet connection, because unstable Wi-Fi is handled by reconnection and resynchronization and not by full offline operation [8, 24]. Open-source components keep the licensing cost low, and their licenses must be respected [11].

Localization is a further requirement, since the interface and the dish explanations must support English and Urdu, including right-to-left text display for Urdu. Finally, the system shall be developed and tested in two-week sprints with the supporting tools named in Chapter 1, which keeps the requirements verifiable throughout the project [13].

2.4 Requirement Gathering Techniques Used

Requirements elicitation is the activity of discovering what the stakeholders need from a system, and common techniques include interviews, questionnaires, document analysis and prototyping [23]. For MezFlow the team combined techniques to obtain comprehensive and accurate requirements. Techniques that have been carried out are marked as done, and techniques that are planned for the requirements phase are marked as planned, so that this document does not claim results that do not yet exist.

2.4.1 Review of Existing Systems and Documents (Done)

Document and competitor analysis was the first technique. The team studied ten existing restaurant products, Toast, Square, mr yum, Lightspeed, TouchBistro, Revel, Epos Now, Flipdish, Menu Tiger and Oracle MICROS Simphony, by reading the public vendor documentation as described in Section 1.2.1, and it analyzed the project proposal and the ordering workflow derived from it [1, 2, 5, 6, 7, 17, 18, 19, 20, 21]. The analysis produced the list of features that existing tools document and the gaps that MezFlow targets, and it was the main source of the functional requirements in Section 2.2 [23].

2.4.2 Prototype Walkthrough (Done / Planned)

Prototyping helps stakeholders react to concrete screens instead of abstract text [23]. The team has prepared an interactive prototype of the main MezFlow screens, which gives the supervisor and prospective users something to react to. Walkthroughs of this prototype with restaurant staff and prospective diners are planned, and the observations will be written into the product backlog and into the next revision of this SRS [13, 23].

2.4.3 Interviews (Planned)

Stakeholder interviews are the primary technique planned to validate and extend the requirements. The team plans structured interviews with open-ended questions with restaurant owners or managers, kitchen staff, waiters and cashiers, to learn their current pain points with ordering, kitchen coordination, billing and table turnover [23]. Number of interviewees: [to be filled after the interviews]. Restaurants or roles interviewed: [to be filled after the interviews]. Dates: [to be filled after the interviews]. After each session the notes will be analyzed to find common requirements and themes [23].

2.4.4 Questionnaires (Planned)

Questionnaires reach a wider group than interviews and give countable answers [23]. A short questionnaire is planned for restaurant customers, asking how often they dine out in groups, how they currently order and pay, whether they would scan a table QR code, and which features they value, such as live order status or dish explanations in Urdu. Number of respondents: [to be filled after the survey]. Summary of results: [to be filled after the survey]. The same questionnaire approach will later be repeated in the usability trial, in which the System Usability Scale is used [14, 23].

2.5 Time Frame

The requirements phase is planned to take 34 days, running through the eight phases shown in Table 2.1, and it combines document analysis, prototype walkthroughs, interviews and questionnaires [23]. The durations are the team's proposed plan, not measured results, and they will be corrected if the actual phase lengths differ; the supervisor should confirm the plan [13].

With the requirements gathered and understood, the team can move with clear direction toward designing, developing and implementing a solution that is tailored to restaurants and

aligned with the goals of MezFlow [11]. The first design work follows this SRS, and the first three modules, Identity and Access Control, Digital Menu and QR Table Access, and Collaborative Table Ordering, are the 25% of the functionality to be presented in the mid-phase review in week 7 [13].

Table 2.1 displays the proposed timeline for the various phases.

Table 2.1: Time Frame

Sr. No	**Phase**	**Planned Duration**
1	Project Requirement Initiation	3 Days
2	Requirement Discovery	4 Days
3	Requirement Expansion	4 Days
4	Requirement Settlement	3 Days
5	Requirement Analysis	7 Days
6	Requirement Validation	3 Days
7	Requirement Specification	6 Days
8	SRS Document	4 Days

MezFlow is designed with real people in mind and aims to be secure, reliable and easy to use. Usability comes first with a simple and filter-driven design, maintainability lets the system accept new features without disrupting existing ones, and availability keeps it ready throughout service hours [34]. Security relies on one-time password sign-in, role-based access and server-side price checks, and performance targets of 3 seconds for the first load and 2 seconds for propagation keep the kitchen and the customers in step. Compatibility with ordinary browsers, accessible text and clear messages complete the specification [4, 36].

References

Reference numbers continue from Chapter 1 so that they match the full thesis.

[1] Toast, Inc. Toast POS and Kitchen Display System [Online]. Available: https://pos.toasttab.com. Last Accessed on October 21, 2026.

[2] Block, Inc. Square for Restaurants [Online]. Available: https://squareup.com/us/en/point-of-sale/restaurants. Last Accessed on October 21, 2026.

[3] ISO/IEC 18004:2015, *Information Technology — Automatic Identification and Data Capture Techniques — QR Code Bar Code Symbology Specification*, 2015.

[4] MDN Web Docs. Progressive Web Apps — Best Practices [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps. Last Accessed on September 25, 2026.

[5] Mr Yum Pty Ltd. Mr Yum — QR Ordering and Payments [Online]. Available: https://mryum.com. Last Accessed on September 25, 2026.

[6] Lightspeed Commerce Inc. Lightspeed Restaurant Features [Online]. Available: https://www.lightspeedhq.com/products/restaurant/features/. Last Accessed on October 21, 2026.

[7] TouchBistro Inc. TouchBistro U.S. Product Guide 2026 [Online]. Available: https://www.touchbistro.com/wp-content/uploads/2024/09/US-TouchBistro-Product-Guide-2026.pdf. Last Accessed on October 21, 2026.

[8] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF, RFC 6455, Dec. 2011 [Online]. Available: https://www.rfc-editor.org/rfc/rfc6455. Last Accessed on September 25, 2026.

[9] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[10] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[11] ISO/IEC/IEEE 29148:2018, *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, 2018.

[12] National Restaurant Association. Restaurant Technology Landscape Report 2024 [Online]. Available: https://go.restaurant.org/rs/078-ZLA-461/images/NatRestAssoc_TechLandscapeReport_2024.pdf. Last Accessed on October 21, 2026.

[13] K. Schwaber and J. Sutherland. (2020, Nov.). The Scrum Guide [Online]. Available: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf. Last Accessed on October 21, 2026.

[14] J. Brooke, "SUS: A quick and dirty usability scale," in *Usability Evaluation in Industry*, P. W. Jordan *et al.*, Eds. London, UK: Taylor & Francis, 1996, pp. 189–194.

[17] Revel Systems. Restaurant POS System [Online]. Available: https://revelsystems.com/pos-systems/restaurant-pos. Last Accessed on October 21, 2026.

[18] Epos Now. Hospitality POS System [Online]. Available: https://www.eposnow.com/au/systems/hospitality-pos/. Last Accessed on October 21, 2026.

[19] Flipdish. Table Ordering and QR Code FAQ [Online]. Available: https://help.flipdish.com/en/articles/9585439-table-ordering-and-qr-code-faq. Last Accessed on October 21, 2026.

[20] Menu Tiger. Features [Online]. Available: https://www.menutiger.com/features. Last Accessed on October 21, 2026.

[21] Oracle. Oracle MICROS Simphony POS [Online]. Available: https://www.oracle.com/food-beverage/restaurant-pos-systems/simphony-pos/. Last Accessed on October 21, 2026.

[22] Oracle. Oracle MICROS Kitchen Display Systems (Data Sheet) [Online]. Available: https://oracle.com/a/ocom/docs/industries/hospitality/oracle-micros-kitchen-display-systems-ds.pdf. Last Accessed on October 21, 2026.

[23] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.

[24] MDN Web Docs. Offline and Background Operation [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Offline_and_background_operation. Last Accessed on September 25, 2026.

[25] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[26] S. Ramírez. FastAPI [Online]. Available: https://fastapi.tiangolo.com. Last Accessed on October 21, 2026.

[28] D. Harel, "Statecharts: A visual formalism for complex systems," *Sci. Comput. Program.*, vol. 8, no. 3, pp. 231–274, Jun. 1987.

[29] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015 [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519. Last Accessed on October 21, 2026.

[30] The PostgreSQL Global Development Group. About PostgreSQL [Online]. Available: https://www.postgresql.org/about/. Last Accessed on October 21, 2026.

[31] Vercel, Inc. Next.js Documentation [Online]. Available: https://nextjs.org/docs. Last Accessed on October 21, 2026.

[34] ISO/IEC 25010:2023, *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model*, 2023.

[35] OWASP Foundation. OWASP Top 10:2025 [Online]. Available: https://top10.owasp.org. Last Accessed on October 22, 2026.

[36] World Wide Web Consortium (W3C). Web Content Accessibility Guidelines (WCAG) 2.2 [Online]. Available: https://www.w3.org/TR/WCAG22/. Last Accessed on October 22, 2026.